

**KSCSC**

Cyber Security Consulting · Offensive Security & Cloud

Security Assessment Report — PAM System

Anonymised work sample · PAM appliance assessment (black/grey-box)

Assessment	PAM appliance security assessment
Prepared by	KSCSC — Krzysztof Świdrak
Client	Redacted (financial institution)
Approach	Black/grey-box
Classification	Confidential — anonymised work sample

This is a redacted work sample. Client identity, service-provider name, the tested product's commercial name, hostnames, IP addresses and account names have been replaced with neutral placeholders. Proof-of-concept fragments are retained to illustrate technique; original screenshots are represented by redacted placeholders. Nothing here identifies the original engagement or client.

Contents

Vulnerability Scoring	3
Executive Summary	3
Methodology and Test Environment	3
Findings	4
Vulnerable JS component — date library [Legacy] — High (CVSS3 7.5)	4
Vulnerable JS component — utility library — High (CVSS3 7.5)	4
No enforcement of a secure password policy for local accounts — High (CVSS3 7.5)	4
No identity verification when changing authentication credentials — Medium (CVSS3 6.7)	4
Vulnerable JS components (bootstrap, jquery, jquery-ui) [EoL] — Medium (CVSS3 6.1)	5
RDP service — NLA not enforced — Medium (CVSS3 5.3)	5
Risky memory management — nvlist assertion (C) — Medium (CVSS3 5.3)	6
Default, guessable account names — Medium (CVSS3 5.3)	6
No output encoding on the API side (potential XSS) — Medium (CVSS3 4.4)	7
Excessive exposure of technical data — SessionViewer role — Medium (CVSS3 4.3)	7
Missing Content-Disposition header — Medium (CVSS3 4.3)	7
Insufficient session-cookie protection — Medium (CVSS3 4.2)	7
Improper access control over Password Changers configuration — Medium (CVSS3 4.1)	7
Missing Content-Security-Policy header — Low (CVSS3 3.4)	8
Missing Cache-Control header — Low (CVSS3 3.1)	8
Missing Permissions-Policy header — Low (CVSS3 3.1)	8
Missing charset in Content-Type — Low (CVSS3 3.1)	8
No clearing of browser session attributes (clean logout) — Low (CVSS3 2.1)	8
Discovered services — Info	8
SSH services — Info (positive note)	9
Appendix — ASVS Table (extract)	10

Security Assessment Report — PAM System

Vulnerability Scoring

Findings are scored using **CVSS3**, **CWE**, and the **STRIDE** model. Severity levels: **HIGH**, **MEDIUM**, **LOW**, and **Info**. Exploitability is classified as Low / Medium / High.

Anonymisation notice. This is a redacted work sample. The client identity, the service provider's name, the commercial product name of the tested system, hostnames, IP addresses, workstation names, and account names have been replaced with neutral placeholders (pam.staging.example, 10.0.0.x, <user>, etc.). Proof-of-concept fragments are retained to illustrate technique. Nothing in this document identifies the original engagement or the client.

Executive Summary

The subject of testing was a commercial **PAM (Privileged Access Management)** system — an appliance acting as the central privileged-access control point for the Client (a financial institution). Testing was performed against a development environment.

Key observations, grouped by area:

- **Access control and data exposure (MEDIUM):** authorisation gaps on selected API endpoints (e.g. Password Changers, Listeners) allow unauthorised users to perform technical reconnaissance of the environment.
- **Session and communication security (MEDIUM/LOW):** the absence of enforced NLA in the RDP protocol and the absence of security headers (e.g. CSP) increase exposure to DoS and script injection.
- **Availability and stability:** the memory-management module contains a vulnerability (nvlist) that can crash the service (DoS) prior to authentication.

Overall risk rating: given the system's role as the central privileged-access control point, the overall risk level is assessed as **CRITICAL**. The most significant threat is the ability to remotely take the service offline (DoS), together with the exploitation of known vulnerabilities in third-party components.

Methodology and Test Environment

Two approaches were used: **automated** (vulnerability scanners with a policy matched to the system type) and **manual** (custom activities designed after analysing the environment). Testing followed a methodology based on, among others, **OWASP ASVS v4 (Level 2)**, **CVSS**, and good practice for privileged-access systems. Testing was performed against a development environment, which may affect the assessment and the availability of some functionality; the final decision on risk value rests with the Client. The scope excluded DoS/DDoS testing and source-code review (other than analysis of EoL components). Because a jump host was used, traffic was port-forwarded — the ports shown in this report may differ from the actual system ports (mainly the 40000–40005 range).

Findings

Vulnerable JS component — date library [Legacy] — High (CVSS3 7.5)

The version of the date library in use has publicly confirmed ReDoS (regular-expression denial-of-service) vulnerabilities. An attacker can exhaust server resources with specially crafted input. The library is in Legacy support.

PoC – vulnerability present in: <https://pam.staging.example/static/js/<date-lib>.js>

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

Library version banner / advisory reference in the loaded bundle

Recommendation. Update to the highest supported version; vendor contact required.

Vulnerable JS component — utility library — High (CVSS3 7.5)

The version of the utility library is vulnerable to arbitrary code injection via its template function (`_.template` builds a function via `new Function(...)` without sanitising the variable).

PoC – vulnerability present in: <https://pam.staging.example/static/js/<util-lib>.js>

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

Vulnerable utility-library version identified in the served script

Recommendation. Update to the highest supported version; vendor contact required.

No enforcement of a secure password policy for local accounts — High (CVSS3 7.5)

The system permits simple, dictionary passwords, offers no real-time strength indicator, and does not verify whether a password appears in public breach databases (e.g. Have I Been Pwned). Modern complexity/entropy rules are not enforced for most users.

PoC:

- 1) Set on account `<user>` a breached password (Administrator123!, present in HIBP)
 - the password was accepted, despite the "Enforce password complexity" option being enabled
- 2) For most accounts (e.g. admin) the strength-enforcement option is disabled.

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

Breached password accepted; complexity toggle shown enabled for the account

Recommendation. Adopt a modern policy aligned with NIST/ASVS: minimum length 12–14 characters, any Unicode characters (no rigid composition rules), no forced expiry (only on breach), verification against stolen-password databases (HIBP), passphrases, and a visual strength meter.

No identity verification when changing authentication credentials — Medium (CVSS3 6.7)

The function to reset/change user passwords by an administrator (including their own) does not require re-verification of the acting party's identity. The lack of re-authentication enables abuse if an administrative session is hijacked (facilitates lateral movement).

```
PoC – password-change flow:  
Users -> <user> -> Authentication Methods -> Edit  
No re-verification of the acting party's identity (TOTP/MFA/password).
```

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

Password change confirmed with no re-authentication prompt

Recommendation. Require the administrator's password or an MFA code (step-up authentication) before confirming a change to another user's password.

Vulnerable JS components (bootstrap, jquery, jquery-ui) [EoL] — Medium (CVSS3 6.1)

The interface loads End-of-Life components with publicly known vulnerabilities.

```
PoC – vulnerabilities present in bundles loaded from https://pam.staging.example/static/js/...
```

Recommendation. Update packages to the highest supported versions; vendor contact required.

RDP service — NLA not enforced — Medium (CVSS3 5.3)

The server offers NLA but does not enforce it. An attacker can negotiate TLS before authentication (downgrade), which at scale enables DoS and undermines non-repudiation.

```
PoC – rdp-sec-check scan over the port forwarded on the jump host:  
./rdp-sec-check.pl pam.staging.example:40003  
Flag NLA_SUPPORTED_BUT_NOT_MANDATED_DOS detected despite the listener being set to NLA mode.  
  
RDP login possible by forcing TLS:  
xfreerdp3 /v:10.0.0.10:40003 /sec:tls /cert:ignore
```

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

rdp-sec-check output and the proxy login panel reached over forced-TLS RDP

Unauthorised access opens the attack surface on the proxy login panel (entering non-existent users and attempting to reach a backend resource). To confirm this as a true positive, a custom Python script was written to force full TLS negotiation without NLA — it demonstrated that the gateway does not terminate the connection at the negotiation stage but instead commits CPU/RAM before verifying identity (correctly, it should reject and enforce NLA). This confirms a **pre-authentication DoS** vulnerability. Additionally, the target RDP server has weak LM settings (Send LM & NTLM responses).

```
import socket, struct
def rdp_probe(ip, port, requested_protocol, name):
    print(f"--- TEST: {name} (request for {hex(requested_protocol)}) ---")
    rdp_neg_req = (
        b"\x03\x00\x00\x13"          # TPKT
        b"\x0e\xe0\x00\x00\x00\x00" # X.224
        b"\x01\x00\x08\x00"          # RDP Neg Req Header
        + struct.pack("<I", requested_protocol))
    s = socket.socket(); s.settimeout(5); s.connect((ip, port)); s.send(rdp_neg_req)
    r = s.recv(1024)
    if len(r) >= 19 and r[11] == 0x02:
        sel = struct.unpack("<I", r[15:19])[0]
        # 0x02 = NLA enforced (secure); 0x01 = TLS only (vulnerable); 0x03 = TLS+NLA
        print("selected:", hex(sel))
    s.close()
    # 0x03 = request for TLS+NLA (standard); 0x01 = request for TLS only (downgrade attack)
    rdp_probe("10.0.0.10", 40003, 0x03, "TLS + NLA")
    rdp_probe("10.0.0.10", 40003, 0x01, "TLS ONLY")
```

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

Script output: server selects TLS-only (0x01) instead of enforcing NLA (0x02)

Recommendation. Enforce NLA (Zero Trust, Defence in Depth) on both the proxy and the target server, using domain accounts only. Contact the vendor to establish why the RDP service accepts TLS negotiation before authentication despite the "NLA enabled" option being active on the listener.

Risky memory management — nvlist assertion (C) — Medium (CVSS3 5.3)

A critical defect was identified in the parameter-list handling module: an assertion violation in `nvlist.c` immediately terminates the process (crash) and can render the PAM service entirely unavailable (DoS). The defect requires no authorisation. In line with the test's assumptions, no further analysis was performed in order to avoid causing a DoS.

PoC – connect to RDP as an anonymous/non-existent user:
`xfreerdp3 /v:10.0.0.10:40003 /sec:tls /cert:ignore`
 On attempting to fill the login fields with random data, the connection drops and the superadministrator log shows:
`(nvlist.c:379:nvlist_find) Assertion failed: ((nvl) != ((void *)0)).`

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

Superadministrator log showing the nvlist assertion crash after the dropped connection

Recommendation. Contact the vendor to fix the code and apply a patch (likely available in a newer release).

Default, guessable account names — Medium (CVSS3 5.3)

Accounts with default names were identified (admin — privileged, viewer). The admin account has no strong authentication methods and is active. A guessable name immediately signals the privilege level and facilitates automated attacks.

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

User list showing default admin / viewer accounts; admin active without strong auth

Recommendation. Rename accounts to less guessable names or deactivate them (they may also serve as honeypots for anomaly detection).

No output encoding on the API side (potential XSS) — Medium (CVSS3 4.4)

The backend API trusts input and does not encode output, returning raw HTML/JS tags (e.g. `<script>alert()</script>`). If rendered without protection, this can lead to code execution in the browser.

```
PoC – vulnerable parameters include:
/api/v1/dashboard-storage/dashboard [many JSON parameters]
/api/v2/internal/me [arbitrary URL parameter]
/api/v2/lang [arbitrary URL parameter]
Example:
https://pam.staging.example/api/v2/internal/me?<param>%3Cimg%20src%3da%20onerror%3dalert(1)%3e
=1
```

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

Raw HTML/JS tags reflected unencoded in the API response body

Recommendation. Apply strict input validation at the API layer (reject unexpected HTML tags) and encode output; optionally deploy a WAF for filtering and virtual patching.

Excessive exposure of technical data — SessionViewer role — Medium (CVSS3 4.3)

A user with the SessionViewer role (session viewing) can, via the API — by freely selecting fields (the fields parameter) — read sensitive configuration parameters of objects that are not presented in the GUI.

```
PoC:
https://pam.staging.example/api/v1/listeners?fields=id,name,blocked,protocol,mode,
listen_ip,listen_port,rdp_nla_enabled,tls_enabled,tls_certificate,
tls_certificate_fingerprint_sha256,...&filter=mode:[gateway,transparent,bastion]
```

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

Listener configuration fields returned to a SessionViewer-role account via the API

Recommendation. Return only the minimal set of fields required for the given role, or restrict API use by roles with limited access.

Missing Content-Disposition header — Medium (CVSS3 4.3)

API v2 responses that contain sensitive data and input parameters reflected in the body lack a Content-Disposition: attachment header, which (as Defence in Depth) would prevent the browser from executing any malicious HTML/JS.

Recommendation. Add the header on the server or reverse proxy for API endpoints in line with the returned content.

Insufficient session-cookie protection — Medium (CVSS3 4.2)

The session cookie lacks the `__Host-` prefix, making it susceptible to being overwritten/injected (Cookie Tossing) by other subdomains.

Recommendation. Rename it to `__Host-<cookie>`, do not set the Domain attribute, set Path=/, and set the Secure flag.

Improper access control over Password Changers configuration — Medium (CVSS3 4.1)

Broken Access Control: a low-privilege user (SessionViewer/Auditor role) can read the full Password Changers configuration in JSON, obtaining information about methodology, administrative accounts, and the infrastructure the system manages.

PoC – while logged in as SessionViewer:
https://pam.staging.example/jsapi/system/password_changers

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

Full Password Changers configuration JSON returned to a low-privilege role

Recommendation. Verify authorisation for the endpoint per the principle of Least Privilege; contact the vendor.

Missing Content-Security-Policy header — Low (CVSS3 3.4)

The server does not send a CSP; the browser trusts any injected resource. The X-Frame-Options and X-XSS-Protection headers may not work in newer browsers.

Recommendation. Deploy a restrictive CSP (e.g. script-src 'self', frame-ancestors 'none'); mitigation via an HTTP proxy/WAF is possible without vendor involvement.

Missing Cache-Control header — Low (CVSS3 3.1)

No header controlling caching of the transmitted content.

Recommendation. Add on the server/reverse proxy, e.g. Cache-Control: no-store, no-cache, must-revalidate, proxy-revalidate.

Missing Permissions-Policy header — Low (CVSS3 3.1)

The server does not specify which browser features (camera, geolocation, etc.) are available to the application.

Recommendation. Add a header restricting privileges, e.g. Permissions-Policy: accelerometer=(), camera=(), geolocation=(), microphone=(), payment=(), usb=().

Missing charset in Content-Type — Low (CVSS3 3.1)

No explicit charset in Content-Type (UTF-8 is the default for JSON), which in edge cases enables attacks based on character misinterpretation.

Recommendation. Dynamically append ; charset=utf-8 to API responses on the server/reverse proxy.

No clearing of browser session attributes (clean logout) — Low (CVSS3 2.1)

After logout, local storage and session attributes (cookies) are not removed; per ASVS, the logout process must be "destructive" to temporary client-side data.

Recommendation. Implement a client-side clean-up mechanism triggered on logout; contact the vendor.

Discovered services — Info

Scan of host 10.0.0.20:
 22/TCP SSH — SSH services
 80/TCP HTTP — Nginx
 443/TCP HTTPS — Nginx (TLS); certificate reveals an internal domain name
 3389/TCP RDP — remote desktop
 65522/TCP SSH — Admin SSH

[ SCREENSHOT REDACTED — PoC evidence held in the original]

nmap service/version scan of the target host

Recommendation. Verify that the open services match the intended architecture; disabling HTTP (port 80) is recommended if there is no justified technical need.

SSH services — Info (positive note)

The SSH services are configured exemplarily (Enterprise/High-Security): resistant to the Terrapin attack (kex-strict), support for a post-quantum algorithm (snt-rup761x25519), and no obsolete algorithms (MD5, SHA1, 3DES, CBC).

Recommendation (hardening). Optionally remove 64-bit MACs (umac-64*) in favour of umac-128/hmac-sha2, and prefer AEAD ciphers (chacha20-poly1305, aes-gcm) over CTR modes — for an "A+" rating.

Appendix — ASVS Table (extract)

The assessment was performed against **OWASP ASVS v4, Level 2**. Below is a representative extract of compliance statuses (the full table is in the original). The black/grey-box scope did not permit verification of requirements needing OS/code access (marked N/A).

ASVS	Area	Status	Note
2.1.5	Password change	Non-compliant	SessionViewer can change passwords
2.1.9	No rigid composition rules	Non-compliant	Upper/lower/digit/special required by default
2.5.4	No default accounts	Non-compliant	admin, viewer
3.2.2	Session token entropy $\geq$ 64 bit	Compliant	32 chars ~165 bit
3.4.2 / 3.4.3	HttpOnly / SameSite	Compliant	
5.2.1	Input sanitisation	Non-compliant	RDP and API do not sanitise some input
5.4.1	Memory safety	Non-compliant	nvlist crash (DoS risk)
7.1.3	Security-event logging	Partial	no trace of TLS-only auth attempts
8.2.1	Anti-cache headers	Non-compliant	no Cache-Control
10.2.2	Excessive permissions	Non-compliant	no Permissions-Policy
11.1.4	Anti-automation	Partial	no alerting for TLS-only RDP
14.2.2	Removal of unneeded components	Non-compliant	EoL JS components
14.4.1	Charset in Content-Type	Non-compliant	not always specified

(Requirements in areas 1.x, 6.x, 12.x marked N/A in the original — out of scope or requiring OS/code access — are omitted from this extract.)